UNIVERSITY OF ENGINEERING AND TECHNOLOGY

MARDAN

Name	**ASIM AYAZ**
Registration No#	**25MDBCS804**
Assignment	**Lab Assignment 9**
Submitted to	**Mam Alishba**
Semester	**2nd**
Date	**11-04-2026**

ASSIGNMENT 09

Q no:1

Implement the concept of pointer to base class object without using virtual keyword. The base and derived class should have only one-member function of your own choice.

Lab8_1.cpp | Lab8_2.cpp | Lab8_3.cpp | Lab8_4.cpp | Lab9_1.cpp

Lab9_1.cpp > ...

```cpp
#include <iostream>
using namespace std;
class Base {
public:
    void show() {
        cout << "Base class show() function called." << endl;
    }
};
class Derived : public Base {
public:
    void show() {
        cout << "Derived class show() function called." << endl;
    }
};
int main() {
    Derived d;
    Base* ptr = &d;
    ptr->show();
    return 0;
}
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code> cd "e:\2nd Semester\1 OOPS\OOP
b9_1 }
Base class show() function called.
PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code>
```

Q no:2

Implement the concept of pointer to base class object using virtual keyword in the base class

Lab9_2.cpp > main()

```cpp
#include <iostream>
using namespace std;
class Base {
public:
        virtual void show() {
            cout << "Base class show() function called." << endl;
        }
};
class Derived : public Base {
public:
        void show() override {
            cout << "Derived class show() function called." << endl   ;
        }
};
int main() {
        Derived d;
        Base* ptr = &d;
        ptr->show();
            return 0;
}
```

Output:

```cpp
class Derived : public Base {
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code> cd "e:\2nd Semester\1 OOPS
b9_2 }
Derived class show() function called.
PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code>
```

Q no: 3

Create a class Counter. It should have only pure virtual functions. It should have a method click() that would advance the counter. It should have methods get() and set() for accessing and mutating the counter's value.

Lab9_3.cpp > main()

```cpp
#include <iostream>
using namespace std;
class Counter {
public:
    virtual void click() = 0;
    virtual int get() const = 0;
    virtual void set(int val) = 0;
    virtual ~Counter() {}
};
class MyCounter : public Counter {
private:
    int count;
public:
    MyCounter() : count(0) {}
    void click() override {
        ++count;
    }
    int get() const override {
        return count;
    }
    void set(int val) override {
        count = val;
```

```
    void set(int val) override {
        count = val;
    }
};
int main() {
    MyCounter c;
    Counter* ptr = &c;
ptr->set(10);
    cout << "Initial value: " << ptr->get() << endl;
 ptr->click();
    ptr->click();
    cout << "After two clicks: " << ptr->get() << endl;
     return 0;
}
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code> cd "e:\2nd Seme
b9_3 }
Initial value: 10
After two clicks: 12
PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code>
```

Q no: 4

From the following scenario in the Assignment.

Lab9_4.cpp > main()

```cpp
#include <iostream>
using namespace std;

class father {
protected:
    int age;
public:
    father(int x) {
        age = x;
    }
    virtual void iam() {
        cout << "I am father, my age is " << age << endl;
    }
    virtual ~father() {}
};

class son : public father {
public:
    son(int x) : father(x) {}
    void iam() override {
        cout << "I am son, my age is " << age << endl;
    }
};
```

```
class daughter : public father {
public:
    daughter(int x) : father(x) {}
    void iam() override {
        cout << "I am daughter, my age is " << age << endl;
    }
};

int main() {
    father f(45);
    son s(20);
    daughter d(18);
    cout << "Direct calls:" << endl;
    f.iam();
    s.iam();
    d.iam();
    cout << "\n= Polymorphism using base pointer =:" << endl;
    father* ptr;
    ptr = &s;
    ptr->iam();
ptr = &d;
    ptr->iam();

    return 0;
}
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code> cd "e:\2nd Semester\1 OOP
b9_4 }
Direct calls:
I am father, my age is 45
I am son, my age is 20
I am daughter, my age is 18

= Polymorphism using base pointer =:
I am son, my age is 20
I am daughter, my age is 18
PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code>
```

Q no: 5

Create a class shape through which other derive classes will create rectangle and triangle shapes. The base class shape has two protected float data attributes, pure virtual function area() and getDimensions(). The two derived classes triangle and rectangle should have member function area() which will calculate area of triangle (0.5 * base * height) and rectangle (length * breadth) respectively. In main function create triangle and rectangle class objects and call getDimensions() function of class shape respectively.

Lab9_5.cpp > Shape > ~Shape()

```cpp
#include <iostream>
using namespace std;
class Shape {
protected:
    float dim1, dim2;
public:
virtual float area() = 0;
      void getDimensions() {
        cout << "Enter first dimension: ";
        cin >> dim1;
            cout << "Enter second dimension: ";
        cin >> dim2;
    }
            virtual ~Shape() {}
};
     class Rectangle : public Shape {
public:
    float area() override {
        return dim1 * dim2;
    }
};
```

```
};
class Triangle : public Shape {
public:
    float area() override {
        return 0.5f * dim1 * dim2;
    }
};
int main() {
    Rectangle rect;
    Triangle tri;
    cout << "=== Rectangle ===" << endl;
    rect.getDimensions();
    cout << "Area of rectangle: " << rect.area() << endl;
    cout << "\n=== Triangle ===" << endl;
    tri.getDimensions();
    cout << "Area of triangle: " << tri.area() << endl;
    return 0;
}
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code> cd "e:\2nd Semester\1 OO
b9_5 }
=== Rectangle ===
Enter first dimension: 2
Enter second dimension: 1
Area of rectangle: 2

=== Triangle ===
Enter first dimension: 3
Enter second dimension: 2
Area of triangle: 3
PS E:\2nd Semester\1 OOPS\OOPS LAB\Lab Code>
```

The End